

DevPipe Help Center — Frequently Asked Questions

Account & Billing

Q: How do I reset my password?

You can reset your password from the login screen by clicking "Forgot password?" and entering the email associated with your DevPipe account. We'll send a reset link that expires after 30 minutes for security purposes. If you don't receive the email within a few minutes, check your spam folder or contact support@devpipe.io.

Q: I forgot my password, what should I do?

Go to the DevPipe login page and select "Forgot password?" to trigger a reset email. Click the link in that email and choose a new password — it must be at least 10 characters with one number and one symbol. If the link has expired, simply request a new one from the same screen.

Q: What pricing tiers does DevPipe offer?

DevPipe offers four tiers: Starter (2,000 build minutes/month), Team (10,000 build minutes/month), Scale (40,000 build minutes/month), and Enterprise (custom build minutes and SLA). All tiers include unlimited deployment environments, but advanced features like the secrets manager and priority build caching are limited to Team and above. You can compare full feature breakdowns on the Pricing page in your dashboard.

Q: How do I upgrade from the Starter plan to Team?

Open your account Settings, select Billing, and click "Change Plan." Choose Team and confirm — the upgrade takes effect immediately and you'll be billed a prorated amount for the remainder of your current cycle. Build minute limits and secrets manager access update automatically once the upgrade completes.

Q: Can I downgrade my subscription?

Yes, you can downgrade at any time from the Billing section of your account Settings. Downgrades take effect at the start of your next billing cycle so you keep access to your current tier's features until then. Note that downgrading from Team or Scale may disable features like advanced build caching if usage exceeds the lower tier's limits.

Q: What happens if I exceed my monthly build minutes?

If you exceed your plan's build minute allowance, DevPipe will pause new pipeline runs until the next billing cycle resets your quota, unless you have overage billing enabled. Overage minutes are billed at a flat per-minute rate listed on your plan's pricing page. You can monitor usage in real time from the Usage tab in your dashboard.

Q: How do I update my billing information or credit card?

Navigate to Settings > Billing > Payment Methods, where you can add, remove, or update your credit card on file. Changes take effect immediately for your next invoice. DevPipe uses Stripe to process payments, so your card details are never stored directly on our servers.

Q: I forgot which email I used to sign up for DevPipe.

If you're unsure which email is linked to your account, try the "Forgot password?" flow with the emails you commonly use — it will only send a reset link if a match is found. You can also contact support@devpipe.io with your organization name and team members can help verify the account. For security reasons we cannot disclose account emails over chat without verification.

Q: How do I cancel my DevPipe subscription?

Go to Settings > Billing and click "Cancel Subscription" at the bottom of the page. Your pipelines and deployment environments will remain active until the end of your current billing period, after which your account reverts to a read-only state. You can reactivate at any time by choosing a new plan.

Q: Do you offer discounts for annual billing or nonprofits?

Yes, switching to annual billing on the Team or Scale tier saves approximately 15% compared to monthly billing. We also offer a 30% nonprofit and education discount — reach out to support@devpipe.io with verification of your organization's status to apply it. Enterprise customers should speak with their account manager about custom contract terms.

Setup & Installation

Q: How do I create my first build pipeline?

After logging in, click "New Pipeline" from your dashboard and connect a repository from GitHub, GitLab, or Bitbucket. DevPipe will scan your repo for a `devpipe.yml` file, or you can use one of our starter templates to generate one automatically. Once configured, your first build will trigger on the next push to your selected branch.

Q: What's the minimum setup required to connect a GitHub repository?

You'll need admin access to the GitHub repository and a DevPipe account on at least the Starter tier. Install the DevPipe GitHub App from the Integrations page, grant it access to the repositories you want to build, and create a pipeline pointing to your default branch. The first build typically completes within a few minutes depending on your build cache state.

Q: How do I install the DevPipe CLI?

Run `curl -sSL https://get.devpipe.io | sh` on macOS or Linux, or download the Windows installer from the Downloads page in your dashboard. After installation, run devpipe login` to authenticate the CLI with your account. The CLI lets you trigger builds, manage deployment environments, and inspect logs without leaving your terminal.`

Q: Where do I find my API key for setup?

Go to Settings > API Keys and click "Generate New Key." Each key is scoped to your organization and can be restricted to specific deployment environments for added security. Store the key securely, as DevPipe only displays it once at creation time.

Q: How do I configure a devpipe.yml file?

A `devpipe.yml` file defines your build pipeline stages — typically build, test, and deploy — along with the runner image and environment variables to use. You can start from one of our language-specific templates (Node.js, Python, Go, Java, and others) available when creating a new pipeline. Refer to the in-app YAML reference panel for a full list of supported keys.

Q: I'm having trouble setting up deployment environments. How do I get started?

Navigate to your pipeline settings and select the "Environments" tab, then click "Add Environment" to create one for staging, production, or any custom name you choose. Each environment can have its own secrets manager entries, approval rules, and deployment targets such as AWS, Azure, or a custom webhook. Once created, reference the environment name in your `devpipe.yml` deploy stage.

Q: How do I add team members to my DevPipe organization?

Go to Settings > Team Members and click "Invite Member," then enter their email and assign a role (Admin, Developer, or Viewer). Invited members receive an email with a link to join your organization and set up their account. Role permissions control access to secrets manager entries and production deployment environments.

Q: What do I need to set up DevPipe with Bitbucket?

Install the DevPipe app from the Bitbucket Marketplace, then authorize it with a workspace admin account. From the Integrations page in DevPipe, select Bitbucket and choose which repositories to enable. Once linked, create a pipeline as you would for any other source provider.

Q: How do I migrate my existing pipelines from another CI/CD tool?

DevPipe provides import tools for Jenkins, CircleCI, and GitHub Actions configurations, accessible from the "Import Pipeline" option when creating a new pipeline. The importer converts most stages automatically, but custom scripts or plugins may need manual adjustment in your `devpipe.yml`. We recommend testing imported pipelines in a staging deployment environment before pointing them at production.

Q: How do I set up build caching for faster builds?

Build caching is enabled by default on Team tier and above; you can configure cache paths under the "Cache" section of your pipeline settings. Specify directories like `node_modules` or `.gradle` to persist between runs, which significantly reduces build times. Cache storage limits depend on your plan, with Scale and Enterprise offering expanded cache quotas.

Features & Usage

Q: What is a build pipeline in DevPipe?

A build pipeline is a defined sequence of stages — such as build, test, and deploy — that runs automatically when triggered by a code push, pull request, or manual action. Pipelines are configured through your `devpipe.yml` file and can be customized with parallel jobs, conditional steps, and environment-specific variables. You can view pipeline run history and logs from the Pipelines tab in your dashboard.

Q: How do deployment environments work?

Deployment environments represent distinct targets — like staging, production, or QA — that your pipeline can deploy to, each with its own configuration, secrets, and approval rules. You can require manual approval before a deploy reaches a production environment to add a safety checkpoint. Environment-specific variables and secrets manager entries are kept isolated from one another.

Q: What is the secrets manager and how do I use it?

The secrets manager lets you securely store sensitive values like API tokens, database credentials, and signing keys, scoped to specific deployment environments. Secrets are encrypted at rest and only injected into pipeline runs as environment variables at execution time. You can manage secrets from the "Secrets" tab within each environment's settings page.

Q: How does build caching improve pipeline speed?

Build caching stores dependency directories and intermediate build artifacts between pipeline runs, so subsequent builds skip redundant downloads and compilation steps. This is especially effective for projects with large dependency trees, such as Node.js or Java applications. Cache hit rates are visible in the build summary for each pipeline run.

Q: Can I run pipelines in parallel?

Yes, DevPipe supports parallel job execution within a single pipeline, which you can configure using the `parallel` key in your `devpipe.yml`. This is useful for running test suites across multiple environments or splitting large test suites into shards to reduce overall build time. Parallel jobs count toward your plan's concurrent build minute usage.

Q: How do I view build pipeline logs?

Click into any pipeline run from the Pipelines tab to see real-time streaming logs for each stage. Logs are retained for 90 days on the Team tier and above, and 30 days on Starter. You can also download full logs as a text file for offline review or sharing with your team.

Q: What is the difference between a manual and automatic deployment trigger?

Automatic triggers deploy to an environment whenever a pipeline successfully completes its build and test stages, typically used for staging environments. Manual triggers require a team member to click "Approve & Deploy" within the deployment environment, which is recommended for production to add a human checkpoint. You can configure trigger types separately for each environment in your pipeline settings.

Q: How do I set environment variables for my build pipeline?

Environment variables can be set at the pipeline level, the deployment environment level, or directly within your `devpipe.yml` file under the `env` key. Sensitive values should be stored in the secrets manager instead of plain environment variables to keep them encrypted. Variables defined at the environment level override pipeline-level defaults during a deploy.

Q: Does DevPipe support scheduled or cron-triggered pipelines?

Yes, you can configure scheduled triggers under the "Triggers" section of your pipeline settings using standard cron syntax. This is commonly used for nightly builds, periodic dependency checks, or recurring deployment environment cleanups. Scheduled runs count toward your monthly build minutes like any other pipeline run.

Q: How can I roll back a deployment?

From the deployment environment's history view, select a previous successful deployment and click "Rollback." DevPipe will redeploy that exact build artifact to the environment without re-running the build pipeline. Rollbacks are logged in the activity history for audit purposes.

Troubleshooting

Q: My build pipeline is stuck in a queued state. What should I do?

Queued builds usually indicate you've reached your plan's concurrent build limit, which varies between Starter, Team, and Scale tiers. Check the Usage tab to confirm current concurrency, and consider upgrading your plan if this happens frequently. If concurrency looks normal, try canceling and re-triggering the pipeline run.

Q: Why is my deployment failing at the approval step?

This typically happens when the approver assigned to the deployment environment hasn't reviewed the request, or their account permissions have changed. Check the Environments tab to confirm the correct approvers are assigned and have Admin or Developer roles. You can also reassign approvers without needing to re-run the pipeline from scratch.

Q: My build cache doesn't seem to be working — builds are slow every time.

First confirm that build caching is enabled in your pipeline settings and that your cache paths in `devpipe.yml` match your project's actual dependency directories. Cache misses can also occur if your runner image changes between builds, since caches are scoped per image version. If the issue persists after verifying both, contact support with your pipeline ID for further investigation.

Q: I'm getting a "secrets manager access denied" error during my build.

This error means the pipeline's deployment environment doesn't have permission to read the specific secret, or the secret was deleted or rotated. Check the Secrets tab for that environment to confirm the secret exists and is correctly referenced in your `devpipe.yml`. Also verify your team member role has sufficient permissions to access secrets in that environment.

Q: Why did my pipeline fail immediately with no logs?

An immediate failure with no logs usually points to a malformed `devpipe.yml` file or an invalid runner image specified in your configuration. Use the YAML validator built into the pipeline editor to catch syntax errors before committing. If the YAML is valid, check that the specified runner image name and tag actually exist.

Q: My GitHub webhook isn't triggering new builds.

Check the Integrations page to confirm the DevPipe GitHub App still has an active connection and the correct repository permissions. Webhooks can also fail silently if GitHub's delivery was blocked — review the "Recent Deliveries" log in your GitHub repo's webhook settings for error codes. Reinstalling the DevPipe GitHub App typically resolves persistent webhook issues.

Q: I can't log in even after resetting my password.

Clear your browser cache and cookies, or try logging in from an incognito window, since stale session tokens can sometimes interfere with login. If the problem continues, confirm you're using the correct organization URL, as DevPipe accounts are tied to specific workspace subdomains. Contact support@devpipe.io if you still can't access your account after these steps.

Q: Why does my Slack integration stop sending notifications?

Slack notifications can stop if the OAuth token connecting DevPipe to your Slack workspace has expired or been revoked. Go to Integrations > Slack and click "Reconnect" to refresh the authorization. Also verify the notification rules for your pipeline or deployment environment haven't been disabled in the project settings.

Q: My build is failing only in the production deployment environment, not staging.

This is often caused by environment-specific variables or secrets manager entries that differ between staging and production. Compare the environment variable lists for both environments under their respective settings tabs to spot mismatches. Differences in approval gates or deployment targets can also cause production-only failures.

Q: I accidentally deleted a pipeline. Can it be restored?

Deleted pipelines are retained in a recoverable state for 14 days before being permanently purged. Contact support@devpipe.io with your organization name and the pipeline name to request restoration within that window. After 14 days, pipeline configuration and history cannot be recovered.

Integrations

Q: Which source control providers does DevPipe support?

DevPipe integrates natively with GitHub, GitLab, and Bitbucket, allowing you to connect repositories and trigger pipelines from pushes, pull requests, or merge requests. Each integration is managed from the Integrations page in your dashboard. Self-hosted GitLab and Bitbucket Server instances are supported on the Scale and Enterprise tiers.

Q: How do I connect DevPipe to Slack?

Go to Integrations > Slack and click "Connect," then authorize DevPipe to access your Slack workspace. Once connected, you can configure which channels receive notifications for build pipeline successes, failures, and deployment approvals. You can set different notification rules per pipeline or deployment environment.

Q: Can I integrate DevPipe with GitLab merge requests?

Yes, after connecting your GitLab account from the Integrations page, DevPipe automatically posts pipeline status checks directly on merge requests. You can configure your pipeline to block merges until required stages pass, similar to branch protection rules. This integration works with both GitLab.com and self-hosted GitLab on eligible plans.

Q: How do I disconnect a Bitbucket repository from DevPipe?

Go to Integrations > Bitbucket, find the repository in the connected list, and click "Disconnect." This stops future builds from being triggered by that repository but does not delete existing pipeline history. You can reconnect the same repository at any time without losing prior configuration.

Q: Does DevPipe support custom webhook integrations?

Yes, you can configure outgoing webhooks under Integrations > Webhooks to notify external systems about pipeline events such as build completion or deployment status changes. Webhooks send a JSON payload to the URL you specify and support custom headers for authentication. This is useful for connecting DevPipe to tools outside our native integration list.

Q: How do I set up Slack notifications for a specific deployment environment only?

Open the deployment environment's settings, go to the "Notifications" tab, and select Slack as the channel, then choose the specific Slack channel to receive alerts. This overrides the pipeline-level default notification settings for that environment only. You can mix and match channels across different environments as needed.

Q: Can I use DevPipe with multiple GitHub organizations?

Yes, you can install the DevPipe GitHub App separately on each GitHub organization you manage and link them all to the same DevPipe account. Repositories from each organization will appear separately under the Integrations page for selection. Build minute usage from all linked organizations counts toward the same plan quota.

Q: I forgot how to reconnect my GitHub account after revoking access.

Go to Integrations > GitHub and click "Connect," which will prompt you to reauthorize the DevPipe GitHub App through GitHub's OAuth flow. Make sure to grant access to the same repositories you previously had enabled to avoid disrupting existing pipelines. If repositories don't appear after reconnecting, check the app's permissions directly in your GitHub organization settings.

Q: Does DevPipe integrate with cloud providers like AWS or Azure?

Yes, DevPipe supports deployment targets on AWS, Azure, and Google Cloud through built-in deploy steps configurable in your `devpipe.yml`. Credentials for these providers should be stored in the

secrets manager and referenced within the relevant deployment environment. Enterprise customers can also request custom deployment target integrations.

Q: How many integrations can I connect on the Starter plan?

The Starter tier allows one source control integration (GitHub, GitLab, or Bitbucket) and one Slack workspace connection. Team and Scale tiers support multiple simultaneous source control and notification integrations. Enterprise plans have no fixed integration limits.

Security & Privacy

Q: How does DevPipe encrypt secrets stored in the secrets manager?

Secrets are encrypted at rest using AES-256 and are only decrypted in memory at the moment a pipeline run needs to inject them as environment variables. Access to secrets is scoped per deployment environment and restricted by team member role. DevPipe never logs secret values, even if a build script accidentally prints them, thanks to automatic log redaction.

Q: Does DevPipe support single sign-on (SSO)?

Yes, SSO via SAML 2.0 is available on the Scale and Enterprise tiers, supporting providers like Okta, Azure AD, and Google Workspace. Once configured, team members can log in using your organization's identity provider instead of a DevPipe-specific password. Contact support@devpipe.io to begin SSO setup for your organization.

Q: Is my source code stored on DevPipe's servers?

DevPipe does not permanently store your source code; repositories are cloned temporarily into an isolated build runner for the duration of a pipeline run and discarded afterward. Build artifacts you explicitly choose to retain are stored encrypted, scoped to your organization. You can review our data handling practices in the Security section of your account settings.

Q: How do I enable two-factor authentication (2FA)?

Go to Settings > Security and click "Enable Two-Factor Authentication," then scan the QR code with an authenticator app like Google Authenticator or Authy. You'll be asked to enter a verification code to confirm setup before it's activated. We strongly recommend enabling 2FA for any account with access to production deployment environments.

Q: What compliance certifications does DevPipe have?

DevPipe maintains SOC 2 Type II compliance and undergoes annual third-party security audits. Enterprise customers can request our latest compliance reports and penetration test summaries through their account manager. GDPR-aligned data processing agreements are available for customers operating in the EU.

Q: Who can access secrets manager entries within my organization?

Access to secrets manager entries is controlled by team member roles and deployment environment permissions — only Admins and Developers explicitly granted access to a given environment can view

or modify its secrets. Viewers can see that a secret exists but cannot reveal its value. All secret access and modification events are logged in the audit trail.

Q: Does DevPipe log or store the contents of my environment variables?

Standard environment variables are visible in pipeline configuration, but any variable stored in the secrets manager is encrypted and redacted from logs automatically. DevPipe's redaction system scans build output for known secret values and masks them before logs are saved. We recommend always using the secrets manager for credentials, tokens, and keys rather than plain environment variables.

Q: How do I view the audit log for my organization?

Go to Settings > Audit Log to see a chronological record of actions like team member changes, secrets manager access, and deployment approvals. Audit logs are retained for 180 days on Team and Scale tiers, and indefinitely on Enterprise. Logs can be exported as CSV for compliance reporting.

Q: What is DevPipe's data retention policy for build artifacts and logs?

Build logs are retained for 30 days on Starter and 90 days on Team and above, while build artifacts you choose to persist follow the retention period set in your pipeline configuration, up to 1 year on Enterprise. Deleted pipelines and their associated data enter a 14-day recovery window before permanent deletion. You can adjust artifact retention settings per pipeline under its Advanced settings.

Q: How do I report a security vulnerability in DevPipe?

We welcome responsible disclosure through our security@devpipe.io mailbox or via our bug bounty program listed on the Security page of our website. Please include reproduction steps and avoid testing against production customer data. Our security team typically acknowledges reports within 48 hours.